Swinburne University of Technology

Faculty of Science, Engineering and Technology

MIDTERM COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: Midterm, Solution Design, Design Pattern, and Iterators
Due date: April 27, 2022, 23:59
Lecturer: Dr. Markus Lumpe

Your name: Le Duc Duyet **Your student ID:** 105243570

Check Tutorial	Mon 10:30	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Tues 16:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30

Marker's comments:

Problem	Marks	Obtained
1	68	
2	120	
3	56	
4	70	
Total	314	

KeyProvider

```cpp
// KeyProvider.cpp
#include "KeyProvider.h"
#include <cstring>

KeyProvider::KeyProvider(const std::string &aKeyword) : fKeyword(nullptr),
fSize(0), fIndex(0)
{
    initialize(aKeyword);
}

KeyProvider::~KeyProvider()
{
    delete[] fKeyword;
}

void KeyProvider::initialize(const std::string &aKeyword)
{
    delete[] fKeyword;
    fSize = aKeyword.length();
    fKeyword = new char[fSize];
    for (size_t i = 0; i < fSize; ++i)
    {
        fKeyword[i] = std::toupper(aKeyword[i]);
    }
    fIndex = 0;
}

char KeyProvider::operator*() const
{
    return fKeyword[fIndex];
}

KeyProvider &KeyProvider::operator<<(char aKeyCharacter)
{
    fKeyword[fIndex] = std::toupper(aKeyCharacter);
    fIndex = (fIndex + 1) % fSize;
    return *this;
}
```

iVigenereStream

```cpp
// iVigenereStream.cpp
#include "iVigenereStream.h"
#include <cstdint>

using namespace std;
```

```
iVigenereStream::iVigenereStream(Cipher aCipher, const std::string &aKeyword,
const char *aFileName)
    : fCipherProvider(aKeyword), fCipher(aCipher)
{
    if (aFileName)
        open(aFileName);
}

iVigenereStream::~iVigenereStream()
{
    close();
}

void iVigenereStream::open(const char *aFileName)
{
    fIStream.open(aFileName, ios::in | ios::binary);
}

void iVigenereStream::close()
{
    fIStream.close();
}

void iVigenereStream::reset()
{
    fCipherProvider.reset();
    seekstart();
}

bool iVigenereStream::good() const
{
    return fIStream.good();
}

bool iVigenereStream::is_open() const
{
    return fIStream.is_open();
}

bool iVigenereStream::eof() const
{
    return fIStream.eof();
}

uint64_t iVigenereStream::position()
{
    return fIStream.tellg();
}
```

```
void iVigenereStream::seekstart()
{
    fIStream.clear();
    fIStream.seekg(0, ios_base::beg);
}

iVigenereStream &iVigenereStream::operator>>(char &aCharacter)
{
    char lChar;
    if (fIStream.get(lChar))
    {
        aCharacter = fCipher(fCipherProvider, lChar);
    }
    else
    {
        fIStream.clear();
    }
    return *this;
}
```

Vigenere

```
// Vigenere.cpp
#include "Vigenere.h"
#include <cctype>

Vigenere::Vigenere(const std::string &aKeyword)
    : fKeyword(aKeyword), fKeywordProvider(aKeyword)
{
    initializeTable();
}

void Vigenere::initializeTable()
{
    for (char row = 0; row < CHARACTERS; row++)
    {
        char lChar = 'A' + row;
        for (char column = 0; column < CHARACTERS; column++)
        {
            fMappingTable[row][column] = lChar;
            if (++lChar > 'Z')
                lChar = 'A';
        }
    }
}

std::string Vigenere::getCurrentKeyword()
{
```

```
    std::string result;
    Vigenere temp(*this);
    for (size_t i = 0; i < fKeyword.length(); ++i)
    {
        result += *temp.fKeywordProvider;
        temp.fKeywordProvider << result.back();
    }
    return result;
}

void Vigenere::reset()
{
    fKeywordProvider.initialize(fKeyword);
}

char Vigenere::encode(char aCharacter)
{
    if (!std::isalpha(aCharacter))
        return aCharacter;
    bool isLower = std::islower(aCharacter);
    char keyChar = *fKeywordProvider;
    char encoded = fMappingTable[keyChar - 'A'][std::toupper(aCharacter) -
'A'];
    fKeywordProvider << aCharacter;
    return isLower ? std::tolower(encoded) : encoded;
}

char Vigenere::decode(char aCharacter)
{
    if (!std::isalpha(aCharacter))
        return aCharacter;
    bool isLower = std::islower(aCharacter);
    char keyChar = *fKeywordProvider;
    int row = keyChar - 'A';
    int col = 0;
    while (fMappingTable[row][col] != std::toupper(aCharacter))
        ++col;
    fKeywordProvider << (col + 'A');
    return isLower ? std::tolower(col + 'A') : col + 'A';
}
```

VigenereForwardIterator

```
// VigenereForwardIterator.cpp
#include "VigenereForwardIterator.h"

VigenereForwardIterator::VigenereForwardIterator(iVigenereStream &aIStream)
    : fIStream(aIStream), fEOF(false)
```

```
{
    ++(*this);
}

char VigenereForwardIterator::operator*() const
{
    return fCurrentChar;
}

VigenereForwardIterator &VigenereForwardIterator::operator++()
{
    if (fIStream >> fCurrentChar)
    {
        fEOF = false;
    }
    else
    {
        fEOF = true;
    }
    return *this;
}

VigenereForwardIterator VigenereForwardIterator::operator++(int)
{
    VigenereForwardIterator temp = *this;
    ++(*this);
    return temp;
}

bool VigenereForwardIterator::operator==(const VigenereForwardIterator
&aOther) const
{
    return (&fIStream == &aOther.fIStream) && (fEOF == aOther.fEOF);
}

bool VigenereForwardIterator::operator!=(const VigenereForwardIterator
&aOther) const
{
    return !(*this == aOther);
}

VigenereForwardIterator VigenereForwardIterator::begin() const
{
    VigenereForwardIterator it(fIStream);
    it.fIStream.reset();
    return it;
}
```

```
VigenereForwardIterator VigenereForwardIterator::end() const
{
    VigenereForwardIterator it(fIStream);
    it.fEOF = true;
    return it;
}
```